Thinker

Thinker is an associative knowledge graph and collaborative LLM wiki built for teams that need a shared digital brain. Instead of asking people to manually categorize documents into rigid folders or write bidirectional links by hand, the platform ingests raw notes and thoughts, extracts core concepts with a local language model, and compiles them into a living graph structure inspired by the LLM wiki pattern proposed by Andrej Karpathy.

By pairing a linked interface (graph that looks like a neural network) with role based access control, the system lets organizations maintain an autonomous second brain where knowledge compounds naturally as users write.

The codebase currently provides the core Go web service, PostgreSQL storage, sqlc generated database access, and server rendered HTML templates. The asynchronous entity extraction worker and graph expansion pipeline represent the immediate architectural roadmap.

Overview

Most note taking platforms rely on folder hierarchies or basic keyword search. When teams scale, these structures quickly break because people organize information differently and maintenance becomes a burden. Standard retrieval augmented generation also falls short over time because it answers questions from scratch by slicing documents into arbitrary chunks without preserving overall context or persistent synthesis.

Thinker treats ideas as an active, compounding artifact:

- Every submitted note or thought becomes an idea node inside a semantic graph.
- A local language model compiles incoming content, extracting entities, identifying topics, and noting connections or contradictions.
- Explicit relationships create edges between ideas, allowing multi hop discovery without relying solely on brittle text matching.
- Role based access control ensures users only discover and traverse knowledge permitted for their organizational role.
- The search experience operates independently of manual tagging by traversing the graph structure and combining text matching with graph expansion.
  The present build contains the foundation for user and idea records, named route handling, database access layer, and server rendered template views.

Stack

- go 1.26.5

> - github.com/gin-contrib/multitemplate v1.1.4
> - github.com/gin-gonic/gin v1.12.0
> - github.com/jackc/pgx/v5 v5.11.0
> - github.com/joho/godotenv v1.5.1

- LLM

> - Qwen2.5-3B-Instruct-GGUF
> - MiniMax M3.1 Flash

- HTML5
- CSS

> - Tailwind 4

- JavaScript

Architecture and data flow



1. Ingestion

- Gin HTTP router
  > Access to a url path via web app.
- RBAC (Role-based access control) middleware
  > Controls permissions for every idea, keyword, and action according to the user role to enforce enterprise privacy.

2. Validation

- Go input type validation handler
  > Validates that submitted files (.pdf, .csv, .txt, .docx) match allowed extensions and size limits before processing.

3. Preprocessing

- Go chunkenizer
  > Splits long documents into manageable, coherent text segments so the local model can ingest them within its context limits.
- Go structs
  > Type-safe memory structures that hold and transform validated raw data across backend layers.
- SQLC payload validation
  > Validates and coerces incoming request parameters into generated database types to reject malformed data before hitting the database.

4. Async dispatch

- Go buffered channel
  > Queues validated document tasks in memory so the HTTP handler can respond immediately without blocking the client.
- Bounded worker pool
  > Limits the number of concurrent background tasks to prevent local CPU and memory exhaustion during heavy inference.

5. Semantic extraction

- llama server
  > High-performance C++ inference engine that exposes a lightweight local HTTP endpoint for the model without Python overhead.
- Qwen 2.5 3B GGUF
  > Fast, low-footprint local language model that reads the text to extract summaries, entities, and conceptual relations.
- Grammar JSON
  > Forces the model output into an exact schema via GBNF grammars to ensure deterministic parsing in Go.

6. Vector embedding

- specialized embedding model
  > Translates extracted concepts and idea texts into dense numerical vectors that capture geometric semantic meaning.

- PGVector vector cords storage and indexation
  > Stores embeddings in PostgreSQL and accelerates similarity searches through HNSW indexing.

7. Linking

- Postgres knn search
  > Queries the nearest neighbor vectors in the database to identify candidate ideas with high semantic similarity.
- Go overlap concept linker
  > Computes intersections between extracted entity tags to reinforce edges that share common named concepts.
- Go bidirectional injector
  > Creates reciprocal graph edges in the database and inserts bidirectional links into markdown documents.

8. Wiki and consulting

- Go graph traversal PPR
  > Executes Personalized PageRank walks across graph edges to retrieve deep, multi-hop related ideas without flat chunking.
- Gin HTML templates
  > Renders server-side web layouts and interactive graph views directly to the browser.
- Postgres knowledge graph
  > The persistent relational store that preserves all nodes, edges, and permission constraints as a compounding single source of truth.

Current Status

The system currently includes:

- Gin HTTP setup and graceful initialization.
- Single pool PostgreSQL connector with sqlc integration.
- User listing through both JSON APIs and HTML pages.
- Basic idea listing views.
- Centralized route directory with reverse template resolution.
- Base layout composition using Tailwind CSS.

- Targeted routing and rendering unit tests.

Next Milestones

- Implement authentication, session handling, and role based access control on notes and graph queries.
- Upgrade the PostgreSQL container setup to support pgvector natively.
- Set up a bounded Go worker pool for asynchronous note ingestion and relationship building.
- Connect the worker pool to a local Qwen 2.5 3B instance for concept extraction and summary generation.
- Create graph edge tables to store typed links and mutual relevance scores.
- Implement graph traversal queries with role aware filters using recursive SQL queries.
- Build an interactive visual graph explorer view in the web interface.

References

1. https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f